

Redes Neuronales Recurrentes

Redes Neuronales Recurrentes

Modelando dependencias temporales y secuencias complejas (RNN, LSTM, GRU)

Redes Neuronales Recurrentes

Modelando dependencias temporales y secuencias complejas (RNN, LSTM, GRU)

Jan Polanco Velasco

¿Por qué el orden
importa?

¿Por qué el orden importa?

El problema de la "Bolsa de Palabras"
(*Bag of Words*)

¿Por qué el orden importa?

El problema de la "Bolsa de Palabras"
(*Bag of Words*)

- Consideremos la siguiente oración incompleta:

"El gato se comió el _____"

¿Por qué el orden importa?

El problema de la "Bolsa de Palabras"
(*Bag of Words*)

- Consideremos la siguiente oración incompleta:

"El gato se comió el _____"

- ¿Cómo sabe un modelo si sigue "pescado" y no "avión"?

¿Por qué el orden importa?

El problema de la "Bolsa de Palabras" (*Bag of Words*)

- Consideremos la siguiente oración incompleta:

"El gato se comió el _____"

- ¿Cómo sabe un modelo si sigue "pescado" y no "avión"?
- **Las redes densas (MLP) tradicionales:**
 - Tratan las entradas como un saco desordenado de características (*i.i.d.*).
 - Pierden completamente la sintaxis, el contexto y la estructura temporal.

¿Por qué el orden importa?

El problema de la "Bolsa de Palabras" (*Bag of Words*)

- Consideremos la siguiente oración incompleta:

"El gato se comió el _____"

- ¿Cómo sabe un modelo si sigue "pescado" y no "avión"?
- **Las redes densas (MLP) tradicionales:**
 - Tratan las entradas como un saco desordenado de características (*i.i.d.*).
 - Pierden completamente la sintaxis, el contexto y la estructura temporal

¿Por qué el orden importa?

El problema de la "Bolsa de Palabras" (*Bag of Words*)

- Consideremos la siguiente oración incompleta:

"El gato se comió el _____"

- ¿Cómo sabe un modelo si sigue "pescado" y no "avión"?
- **Las redes densas (MLP) tradicionales:**
 - Tratan las entradas como un saco desordenado de características (*i.i.d.*).
 - Pierden completamente la sintaxis, el contexto y la estructura temporal

Mismo conjunto de palabras, significado opuesto:

1. "No estaba mal, estaba **excelente**." → **Positivo (+1)**

2. "No estaba excelente, estaba **mal**." → **Negativo (-1)**

Una red densa cuenta las palabras idénticas y arroja la misma predicción.
Una **RNN** procesa palabra por palabra manteniendo un vector de memoria interna.

La Neurona Recurrente Básica

La Neurona Recurrente Básica

Desenrollado Temporal (*Unfolding
through time*)

La Neurona Recurrente Básica

Desenrollado Temporal (*Unfolding through time*)

- **Vector de Estado Oculto (h_t):** Representa la memoria acumulada hasta el instante t .

La Neurona Recurrente Básica

Desenrollado Temporal (*Unfolding through time*)

- **Vector de Estado Oculto (h_t):** Representa la memoria acumulada hasta el instante t .
- **Condición inicial:** En el instante cero $h_0 = \mathbf{0}$ (sin recuerdos previos).

La Neurona Recurrente Básica

Desenrollado Temporal (*Unfolding through time*)

- **Vector de Estado Oculto (h_t):** Representa la memoria acumulada hasta el instante t .
- **Condición inicial:** En el instante cero, $h_0 = \mathbf{0}$ (sin recuerdos previos).
- **Ecuaciones fundamentales en cada paso t :**

$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$$

$$\hat{y}_t = g(W_{hy}h_t + b_y)$$

La Neurona Recurrente Básica

Desenrollado Temporal (*Unfolding through time*)

- **Vector de Estado Oculto (h_t):** Representa la memoria acumulada hasta el instante t .
- **Condición inicial:** En el instante cero, $h_0 = \mathbf{0}$ (sin recuerdos previos).
- **Ecuaciones fundamentales en cada paso t :**

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$\hat{y}_t = g(W_{hy}h_t + b_y)$$

- **Pesos compartidos:** Las matrices W_{xh} , W_{hh} , W_{hy} son **las mismas** en cada paso del tiempo.

La Neurona Recurrente Básica

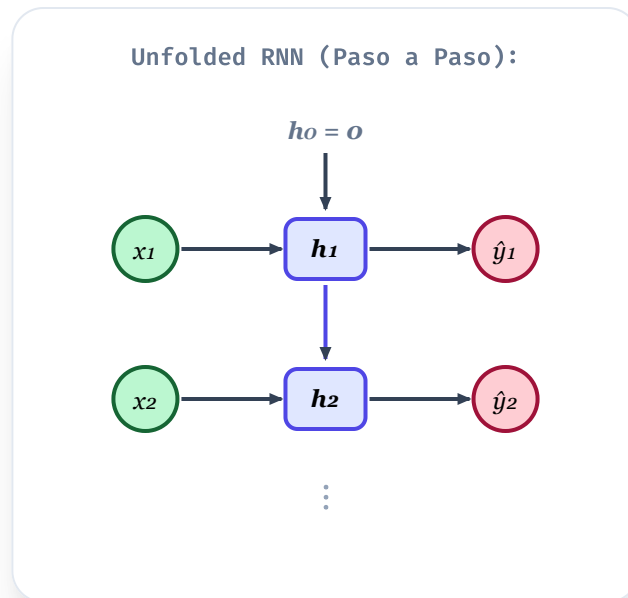
Desenrollado Temporal (*Unfolding through time*)

- **Vector de Estado Oculto (h_t):** Representa la memoria acumulada hasta el instante t .
- **Condición inicial:** En el instante cero $h_0 = \mathbf{0}$ (sin recuerdos previos).
- **Ecuaciones fundamentales en cada paso t :**

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

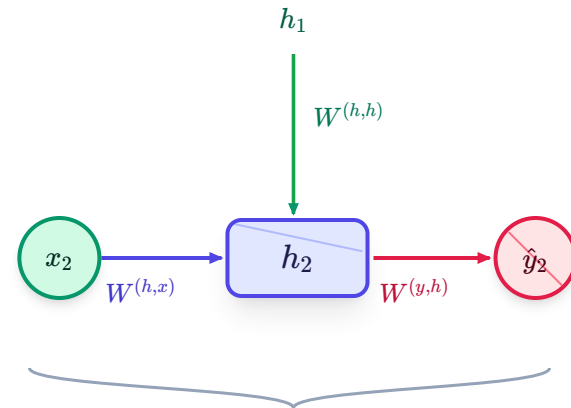
$$\hat{y}_t = g(W_{hy}h_t + b_y)$$

- **Pesos compartidos:** Las matrices W_{xh} , W_{hh} , W_{hy} son **las mismas** en cada paso del tiempo.



1. Vista Celda (Paso t=2)

2. Zoom Neuronal (Fully Connected)

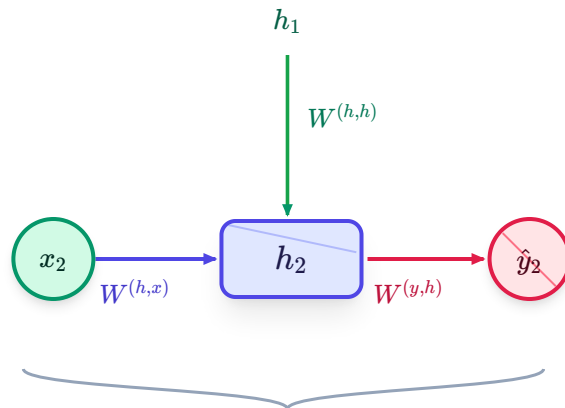


En el paso 2 coinciden 3 matrices de pesos: $W^{(h,x)}$ (entrada), $W^{(h,h)}$ (memoria) y $W^{(y,h)}$ (salida).

Anatomía Interna de la RNN

1. Vista Celda (Paso $t=2$)

2. Zoom Neuronal (Fully Connected)



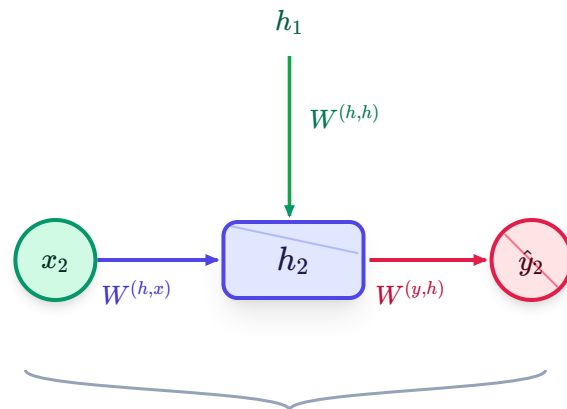
En el paso 2 coinciden 3 matrices de pesos: $W^{(h,x)}$ (entrada), $W^{(h,h)}$ (memoria) y $W^{(y,h)}$ (salida).

Anatomía Interna de la RNN

Zoom al paso temporal $t = 2$: ¿Qué hay dentro de h_2 ?

1. Vista Celda (Paso $t=2$)

2. Zoom Neuronal (Fully Connected)



En el paso 2 coinciden 3 matrices de pesos: $W^{(h,x)}$ (entrada), $W^{(h,h)}$ (memoria) y $W^{(y,h)}$ (salida).

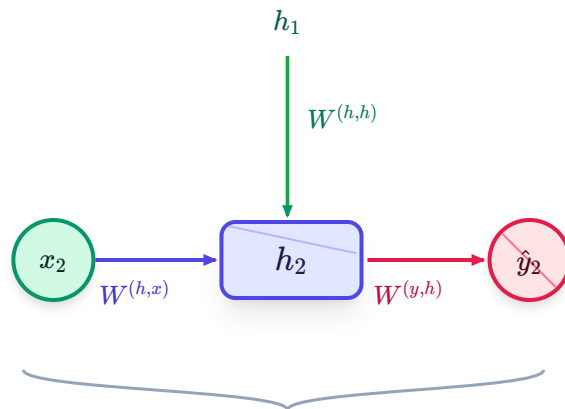
Anatomía Interna de la RNN

Zoom al paso temporal $t = 2$: ¿Qué hay dentro de h_2 ?

- La celda no es un escalar: es una **capa densa** de M neuronas ocultas.

1. Vista Celda (Paso $t=2$)

2. Zoom Neuronal (Fully Connected)



En el paso 2 coinciden 3 matrices de pesos: $W^{(h,x)}$ (entrada), $W^{(h,h)}$ (memoria) y $W^{(y,h)}$ (salida).

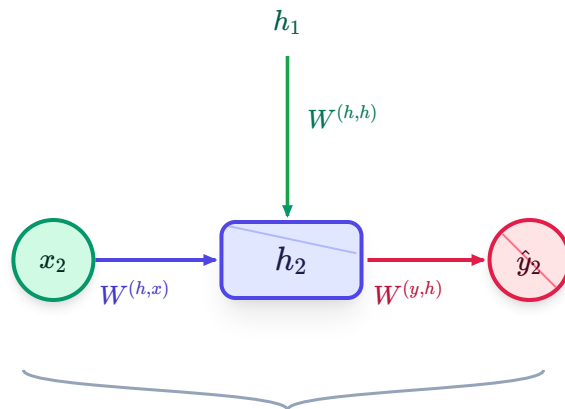
Anatomía Interna de la RNN

Zoom al paso temporal $t = 2$: ¿Qué hay dentro de h_2 ?

- **La celda no es un escalar**: es una **capa densa** de M neuronas ocultas.
- **Dos entradas simultáneas**:
 1. Memoria previa ($h_1 \in \mathbb{R}^M$) ponderada por $W^{(h,h)}$.
 2. Entrada actual ($x_2 \in \mathbb{R}^P$) ponderada por $W^{(h,x)}$.

1. Vista Celda (Paso $t=2$)

2. Zoom Neuronal (Fully Connected)



En el paso 2 coinciden 3 matrices de pesos: $W^{(h,x)}$ (entrada), $W^{(h,h)}$ (memoria) y $W^{(y,h)}$ (salida).

Anatomía Interna de la RNN

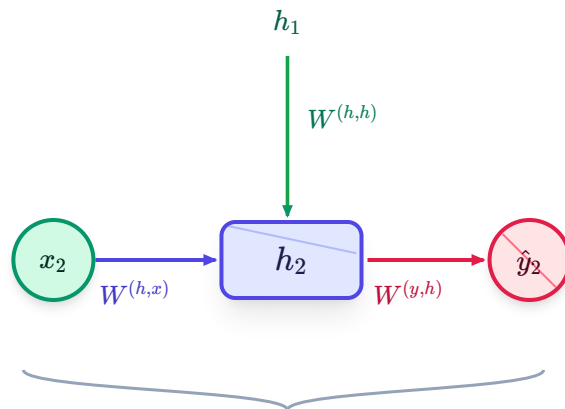
Zoom al paso temporal $t = 2$: ¿Qué hay dentro de h_2 ?

- La celda no es un escalar: es una **capa densa** de M neuronas ocultas.
- Dos entradas simultáneas:
 - Memoria previa ($h_1 \in \mathbb{R}^M$) ponderada por $W^{(h,h)}$.
 - Entrada actual ($x_2 \in \mathbb{R}^P$) ponderada por $W^{(h,x)}$.
- Ecuación escalar por neurona m :

$$h_{2,m} = g \left(\sum_{k=1}^M w_{m,k}^{(h,h)} h_{1,k} + \sum_{p=1}^P w_{m,p}^{(h,x)} x_{2,p} + b_h \right)$$

1. Vista Celda (Paso $t=2$)

2. Zoom Neuronal (Fully Connected)



En el paso 2 coinciden 3 matrices de pesos: $W^{(h,x)}$ (entrada), $W^{(h,h)}$ (memoria) y $W^{(y,h)}$ (salida).

Anatomía Interna de la RNN

Zoom al paso temporal $t = 2$: ¿Qué hay dentro de h_2 ?

- La celda no es un escalar: es una **capa densa** de M neuronas ocultas.
- Dos entradas simultáneas:
 - Memoria previa ($h_1 \in \mathbb{R}^M$) ponderada por $W^{(h,h)}$.
 - Entrada actual ($x_2 \in \mathbb{R}^P$) ponderada por $W^{(h,x)}$.
- Ecuación escalar por neurona m :

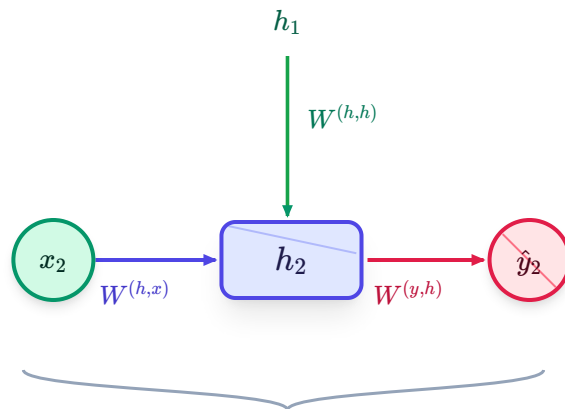
$$h_{2,m} = g \left(\sum_{k=1}^M w_{m,k}^{(h,h)} h_{1,k} + \sum_{p=1}^P w_{m,p}^{(h,x)} x_{2,p} + b_h \right)$$

- Predicción $\hat{y}_2$: combina las M activaciones con $W^{(y,h)}$:

$$\hat{y}_2 = g \left(\sum_{m=1}^M w_m^{(y,h)} h_{2,m} + b_y \right)$$

1. Vista Celda (Paso $t=2$)

2. Zoom Neuronal (Fully Connected)



En el paso 2 coinciden 3 matrices de pesos: $W^{(h,x)}$ (entrada), $W^{(h,h)}$ (memoria) y $W^{(y,h)}$ (salida).

Patrones de Uso en Secuencias

Patrones de Uso en Secuencias

Taxonomía de Arquitecturas
Recurrentes

Patrones de Uso en Secuencias

Taxonomía de Arquitecturas Recurrentes

- **Many-to-One:** Secuencia → una predicción (*sentimientos, audios*).

Patrones de Uso en Secuencias

Taxonomía de Arquitecturas Recurrentes

- **Many-to-One:** Secuencia → una predicción (*sentimientos, audios*).
- **One-to-Many:** Una semilla → secuencia (*música, subtítulos*).

Patrones de Uso en Secuencias

Taxonomía de Arquitecturas Recurrentes

- **Many-to-One:** Secuencia $\rightarrow$ una predicción (*sentimientos, audios*).
- **One-to-Many:** Una semilla $\rightarrow$ secuencia (*música, subtítulos*).
- **Many-to-Many Sincronizado** ($T_x = T_y$): salida por instante (*POS tagging, series*).

Patrones de Uso en Secuencias

Taxonomía de Arquitecturas Recurrentes

- **Many-to-One:** Secuencia $\rightarrow$ una predicción (*sentimientos, audios*).
- **One-to-Many:** Una semilla $\rightarrow$ secuencia (*música, subtítulos*).
- **Many-to-Many Sincronizado** ($T_x = T_y$): salida por instante (*POS tagging, series*).
- **Many-to-Many Asíncrono** ($T_x \neq T_y$): Encoder-Decoder (*traducción, resúmenes*).

Patrones de Uso en Secuencias

Taxonomía de Arquitecturas Recurrentes

- **Many-to-One:** Secuencia $\rightarrow$ una predicción (*sentimientos, audios*).
- **One-to-Many:** Una semilla $\rightarrow$ secuencia (*música, subtítulos*).
- **Many-to-Many Sincronizado** ($T_x = T_y$): salida por instante (*POS tagging, series*).
- **Many-to-Many Asíncrono** ($T_x \neq T_y$): Encoder-Decoder (*traducción, resúmenes*).



El Talón de Aquiles de la RNN Simple

El Talón de Aquiles de la RNN Simple

Desvanecimiento del Gradiente
(*Vanishing Gradient*)

El Talón de Aquiles de la RNN Simple

Desvanecimiento del Gradiente (*Vanishing Gradient*)

- **BPTT:** para ajustar los pesos, el gradiente se multiplica hacia atrás en cada paso:

$$\frac{\partial \mathcal{L}_T}{\partial h_1} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{j=2}^T \frac{\partial h_j}{\partial h_{j-1}}$$

El Talón de Aquiles de la RNN Simple

Desvanecimiento del Gradiente (*Vanishing Gradient*)

- **BPTT:** para ajustar los pesos, el gradiente se multiplica hacia atrás en cada paso:

$$\frac{\partial \mathcal{L}_T}{\partial h_1} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{j=2}^T \frac{\partial h_j}{\partial h_{j-1}}$$

- **Producto de derivadas:** si $|W_{hh}| < 1$ o $\tanh' < 1$, el gradiente decae a **cero**.

El Talón de Aquiles de la RNN Simple

Desvanecimiento del Gradiente (*Vanishing Gradient*)

- **BPTT:** para ajustar los pesos, el gradiente se multiplica hacia atrás en cada paso:

$$\frac{\partial \mathcal{L}_T}{\partial h_1} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{j=2}^T \frac{\partial h_j}{\partial h_{j-1}}$$

- **Producto de derivadas:** si $|W_{hh}| < 1$ o $\tanh' < 1$, el gradiente decae a **cero**.
- **Consecuencia: amnesia a corto plazo** – no recuerda lo visto 8-10 pasos atrás.

El Talón de Aquiles de la RNN Simple

Desvanecimiento del Gradiente (*Vanishing Gradient*)

- **BPTT:** para ajustar los pesos, el gradiente se multiplica hacia atrás en cada paso:

$$\frac{\partial \mathcal{L}_T}{\partial h_1} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{j=2}^T \frac{\partial h_j}{\partial h_{j-1}}$$

- **Producto de derivadas:** si $|W_{hh}| < 1$ o $\tanh' < 1$, el gradiente decae a **cero**.
- **Consecuencia: amnesia a corto plazo** – no recuerda lo visto 8-10 pasos atrás.

⚠ **Pérdida de Dependencias a Largo Plazo**

"Los **perros** que vi corriendo detrás de los niños estaban / estaba cansados."

Al llegar al verbo final, el sujeto plural ya se desvaneció del gradiente.

LSTM (Long Short-Term Memory)

LSTM (Long Short-Term Memory)

Hochreiter & Schmidhuber (1997)

LSTM (Long Short-Term Memory)

Hochreiter & Schmidhuber (1997)

- **La Celda de Memoria:** Diseñada explícitamente para mantener información a través de cientos de pasos.

LSTM (Long Short-Term Memory)

Hochreiter & Schmidhuber (1997)

- **La Celda de Memoria:** Diseñada explícitamente para mantener información a través de cientos de pasos.
- **La Autopista de Estado Celular (C_t):**
 - Flujo de información lineal con muy pocas transformaciones no lineales.
 - El gradiente fluye hacia atrás mediante sumas sin desvanecerse.

LSTM (Long Short-Term Memory)

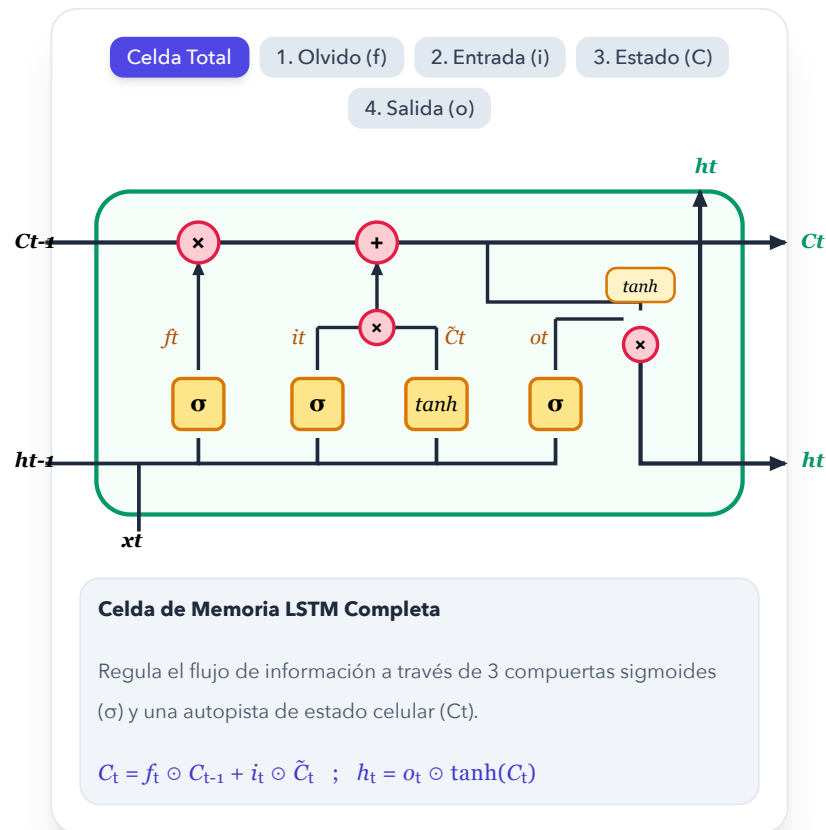
Hochreiter & Schmidhuber (1997)

- **La Celda de Memoria:** Diseñada explícitamente para mantener información a través de cientos de pasos.
- **La Autopista de Estado Celular (C_t):**
 - Flujo de información lineal con muy pocas transformaciones no lineales.
 - El gradiente fluye hacia atrás mediante sumas sin desvanecerse.
- **Las 3 Compuertas Moduladoras (σ):**
 1. **Compuerta de Olvido (f_t):** Qué descartar del pasado.
 2. **Compuerta de Entrada (i_t):** Qué nuevo conocimiento registrar.
 3. **Compuerta de Salida (o_t):** Qué enviar al estado oculto actual h_t .

LSTM (Long Short-Term Memory)

Hochreiter & Schmidhuber (1997)

- **La Celda de Memoria:** Diseñada explícitamente para mantener información a través de cientos de pasos.
- **La Autopista de Estado Celular (C_t):**
 - Flujo de información lineal con muy pocas transformaciones no lineales.
 - El gradiente fluye hacia atrás mediante sumas sin desvanecerse.
- **Las 3 Compuertas Moduladoras (σ):**
 1. **Compuerta de Olvido (f_t):** Qué descartar del pasado.
 2. **Compuerta de Entrada (i_t):** Qué nuevo conocimiento registrar.
 3. **Compuerta de Salida (o_t):** Qué enviar al estado oculto actual h_t .



GRU: Gated Recurrent Unit

GRU: Gated Recurrent Unit

Cho et al. (2014) – La alternativa simplificada

GRU: Gated Recurrent Unit

Cho et al. (2014) – La alternativa simplificada

- **Motivación:** LSTM es potente pero costosa (4 matrices por celda).

GRU: Gated Recurrent Unit

Cho et al. (2014) – La alternativa simplificada

- **Motivación:** LSTM es potente pero costosa (4 matrices por celda).
- **Fusión de estados:** C_t y h_t en una sola variable: h_t .

GRU: Gated Recurrent Unit

Cho et al. (2014) – La alternativa simplificada

- **Motivación:** LSTM es potente pero costosa (4 matrices por celda).
- **Fusión de estados:** C_t y h_t en una sola variable: h_t .
- **Solo 2 compuertas:**
 - Γ_u (**Update**): conservar o actualizar el estado.
 - Γ_r (**Reset**): cuánto influye el pasado en $\tilde{h}_t$.

GRU: Gated Recurrent Unit

Cho et al. (2014) – La alternativa simplificada

- **Motivación:** LSTM es potente pero costosa (4 matrices por celda).
- **Fusión de estados:** C_t y h_t en una sola variable: $\underline{h_t}$.
- **Solo 2 compuertas:**
 - Γ_u (**Update**): conservar o actualizar el estado.
 - Γ_r (**Reset**): cuánto influye el pasado en $\tilde{h}_t$.
- **Actualización convexa:**

$$h_t = (1 - \Gamma_u) \odot h_{t-1} + \Gamma_u \odot \tilde{h}_t$$

GRU: Gated Recurrent Unit

Cho et al. (2014) – La alternativa simplificada

- **Motivación:** LSTM es potente pero costosa (4 matrices por celda).
- **Fusión de estados:** C_t y h_t en una sola variable: h_t .
- **Solo 2 compuertas:**
 - Γ_u (**Update**): conservar o actualizar el estado.
 - Γ_r (**Reset**): cuánto influye el pasado en $\tilde{h}_t$.
- **Actualización convexa:**

$$h_t = (1 - \Gamma_u) \odot h_{t-1} + \Gamma_u \odot \tilde{h}_t$$

LSTM vs. GRU

Eficiencia

LSTM

3 Compuertas (f, i, o)
2 Estados (c, h)

4 × d² Parámetros

GRU

2 Compuertas (r, u)
1 Estado (h)

3 × d² Parámetros (25% menos)

GRU converge significativamente más rápido en conjuntos de datos medianos o con recursos limitados.

Redes Recurrentes Bidireccionales

Redes Recurrentes Bidireccionales

Pasado, Presente y Futuro simultáneos

Redes Recurrentes Bidireccionales

Pasado, Presente y Futuro simultáneos

- **Limitación unidireccional:** Una RNN tradicional en el paso t solo conoce información anterior $(x_1, \dots, x_t)$.

Redes Recurrentes Bidireccionales

Pasado, Presente y Futuro simultáneos

- **Limitación unidireccional:** Una RNN tradicional en el paso t solo conoce información anterior $(x_1, \dots, x_t)$.
- **Solución Bidireccional (BiRNN / BiLSTM):**
 - Capa hacia adelante ($\vec{h}_t$): Lee la secuencia de izquierda a derecha.
 - Capa hacia atrás ($\overleftarrow{h}_t$): Lee la secuencia de derecha a izquierda.

Redes Recurrentes Bidireccionales

Pasado, Presente y Futuro simultáneos

- **Limitación unidireccional:** Una RNN tradicional en el paso t solo conoce información anterior $(x_1, \dots, x_t)$.
- **Solución Bidireccional (BiRNN / BiLSTM):**
 - Capa hacia adelante ($\vec{h}_t$): Lee la secuencia de izquierda a derecha.
 - Capa hacia atrás ($\overleftarrow{h}_t$): Lee la secuencia de derecha a izquierda.
- **Salida combinada:**

$$\hat{y}_t = g \left(W_y [\vec{h}_t, \overleftarrow{h}_t] + b_y \right)$$

Redes Recurrentes Bidireccionales

Pasado, Presente y Futuro simultáneos

- **Limitación unidireccional:** Una RNN tradicional en el paso t solo conoce información anterior ($x_1, \dots, x_t$).
- **Solución Bidireccional (BiRNN / BiLSTM):**
 - Capa hacia adelante ($\vec{h}_t$): Lee la secuencia de izquierda a derecha.
 - Capa hacia atrás ($\overleftarrow{h}_t$): Lee la secuencia de derecha a izquierda.
- **Salida combinada:**

$$\hat{y}_t = g \left(W_y [\vec{h}_t, \overleftarrow{h}_t] + b_y \right)$$

- **Requisito fundamental:** Toda la secuencia debe estar disponible con antelación (*no aplicable a predicción financiera en tiempo real*).

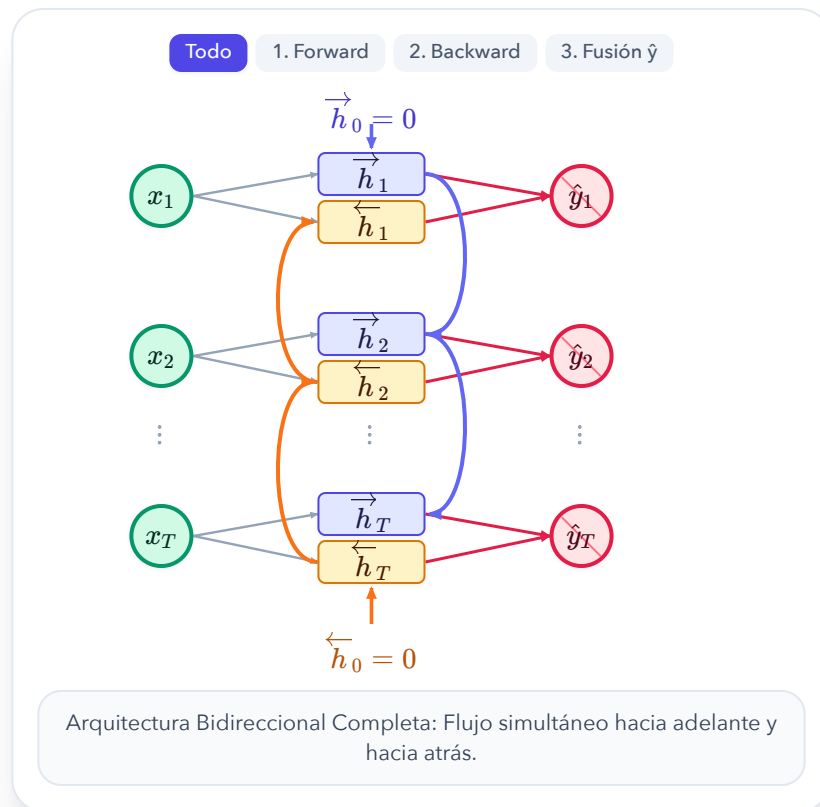
Redes Recurrentes Bidireccionales

Pasado, Presente y Futuro simultáneos

- **Limitación unidireccional:** Una RNN tradicional en el paso t solo conoce información anterior ($x_1, \dots, x_t$).
- **Solución Bidireccional (BiRNN / BiLSTM):**
 - Capa hacia adelante ($\vec{h}_t$): Lee la secuencia de izquierda a derecha.
 - Capa hacia atrás ($\overleftarrow{h}_t$): Lee la secuencia de derecha a izquierda.
- **Salida combinada:**

$$\hat{y}_t = g \left(W_y [\vec{h}_t, \overleftarrow{h}_t] + b_y \right)$$

- **Requisito fundamental:** Toda la secuencia debe estar disponible con antelación (*no aplicable a predicción financiera en tiempo real*).



Desafíos de LSTM

- **Naturaleza estrictamente secuencial:** No se pueden paralelizar en GPU durante el entrenamiento.
- **Degradación en secuencias extensas:** En secuencias de más de 500 pasos, el cuello de botella vectorial olvida detalles finos.

Complejidad temporal: $O(T)$

¿Dónde siguen ganando?

- **Dispositivos embebidos y Edge AI:** Consumen una fracción insignificante de memoria RAM frente a un Transformer.
- **Inferencia paso a paso en tiempo real:** Streaming de audio o sensores industriales de bajo costo.
- **Bajo volumen de datos:** No requieren millones de parámetros pre-entrenados para converger.

Huella de memoria: Mínima

La Era Transformer

El mecanismo de **Atención (Self-Attention)** permite conectar cualquier par de palabras directamente en **$O(1)$** operaciones sin depender de un estado recurrente.

Base de GPT, Llama y BERT

¿Siguen siendo relevantes las LSTM frente a Transformers?

Desafíos de LSTM

- **Naturaleza estrictamente secuencial:** No se pueden paralelizar en GPU durante el entrenamiento.
- **Degradación en secuencias extensas:** En secuencias de más de 500 pasos, el cuello de botella vectorial olvida detalles finos.

Complejidad temporal: $O(T)$

¿Dónde siguen ganando?

- **Dispositivos embebidos y Edge AI:** Consumen una fracción insignificante de memoria RAM frente a un Transformer.
- **Inferencia paso a paso en tiempo real:** Streaming de audio o sensores industriales de bajo costo.
- **Bajo volumen de datos:** No requieren millones de parámetros pre-entrenados para converger.

Huella de memoria: Mínima

La Era Transformer

El mecanismo de **Atención (Self-Attention)** permite conectar cualquier par de palabras directamente en $O(1)$ operaciones sin depender de un estado recurrente.

Base de GPT, Llama y BERT

De la teoría a la práctica en Python con TensorFlow / Keras

1

Modelado de Series Temporales con LSTM

Manejo del tensor tridimensional [samples, time_steps, features]

2

Benchmarking: Simple RNN vs. LSTM vs. GRU

Comparación de velocidad de convergencia, loss y número de parámetros

3

Inferencia Recurrente y Proyecciones Futuras

Evaluación de estabilidad y retención de memoria a largo plazo



Laboratorio Práctico de la Sesión 2

De la teoría a la práctica en Python con TensorFlow / Keras

1

Modelado de Series Temporales con LSTM

Manejo del tensor tridimensional [samples, time_steps, features]

2

Benchmarking: Simple RNN vs. LSTM vs. GRU

Comparación de velocidad de convergencia, loss y número de parámetros

3

Inferencia Recurrente y Proyecciones Futuras

Evaluación de estabilidad y retención de memoria a largo plazo